

Mission 1 — Find AI Around You

3-row table

System / feature I use	What I think it does	Is AI/ML actually involved?
YouTube recommendations	Recommends videos based on my interests, viewing history and interactions.	Yes — likely AI/ML. Recommendation systems use data about user behaviour to predict what content may be relevant.
Google Maps	Predicts travel time and helps recommend routes based on traffic and other conditions.	Yes — AI/ML is used. It can use large amounts of traffic and location data to make predictions.
Gmail Spam Filter	Classifies incoming emails as normal, spam, phishing, etc.	Yes — verified. Google states that Gmail uses machine learning for spam detection.

Evidence for one example: Gmail

I chose **Gmail's spam filter**.

Google has explicitly documented that Gmail uses **machine learning algorithms** to detect spam. Google explains that Gmail historically used manually created rules, but later added machine-learning systems that identify patterns and predict which emails are likely to be spam.

Google also currently describes Gmail's defences as **AI-powered**, saying they protect users from spam, phishing and malware.

Could a simple rule-based program do something similar?

Yes, to some extent.

A simple rule-based system could say:

- If an email contains certain suspicious words → mark as spam.
- If the sender is blocked → send to spam.
- If an email contains certain suspicious patterns → flag it.

In fact, Google says Gmail **historically used a rule-based system** for spam detection.

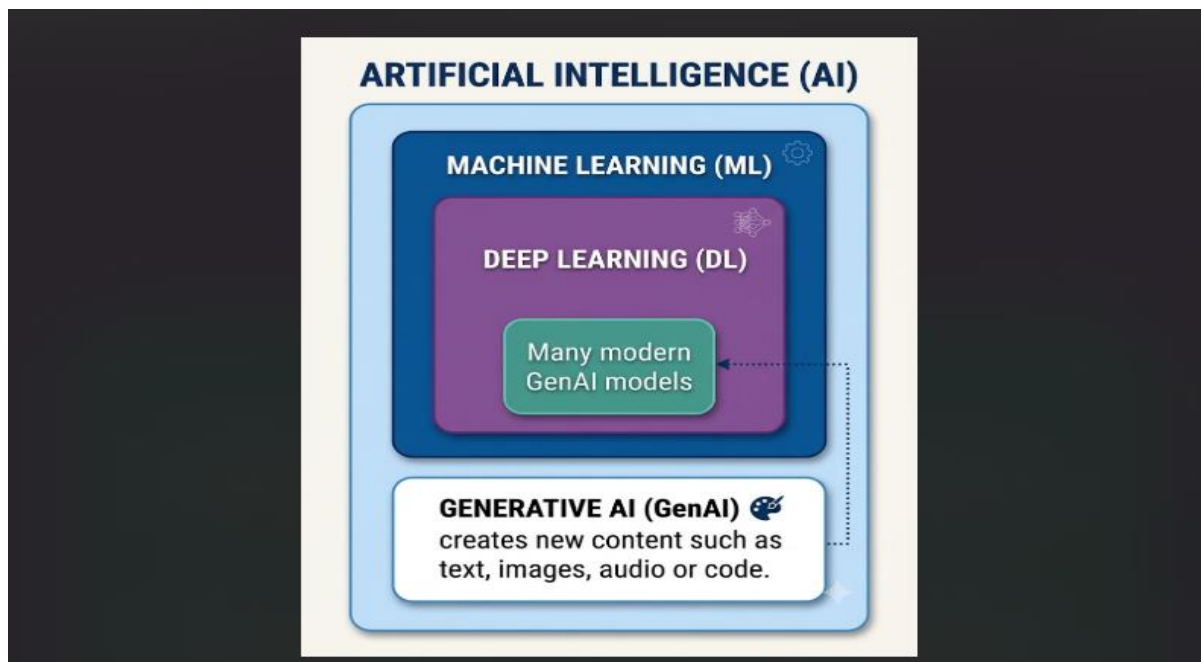
However, a rule-based system would be less flexible. Modern spam changes constantly, while machine learning can identify new patterns and adapt more effectively. Google specifically says its machine-learning approach can find new patterns faster than the previous manual rule-based approach.

Short conclusion

AI is already part of many technologies I use every day, even when I do not directly notice it. YouTube and Google Maps can use AI/ML to make recommendations and predictions, while Gmail uses machine learning to classify spam. This shows that AI is not only chatbots like ChatGPT; it is already built into ordinary digital services.

Mission 2 — AI, ML, Deep Learning & Generative AI

Simple diagram



Important clarification

The diagram is a **simplified way to show the relationship**. AI is the broadest concept. ML is one way of building AI systems, and deep learning is a type of ML.

Generative AI is different because it describes what an AI system does—generates new content—rather than simply being another level inside the AI → ML → Deep Learning hierarchy. Most modern GenAI systems, such as large language models, are built using deep learning.

Four short explanations

1. Artificial Intelligence (AI)

AI is the broad idea of making computers perform tasks that normally require human-like intelligence, such as understanding language, recognizing objects, making decisions, or solving problems.

Example: A navigation system choosing a route based on traffic conditions.

2. Machine Learning (ML)

Machine learning is a way of building AI systems that learn patterns from data instead of being given every rule manually.

For example, instead of programming every possible type of spam email, we can train a system using many examples of spam and non-spam emails.

Example: Gmail's spam detection system learning to identify unwanted emails.

3. Deep Learning (DL)

Deep learning is a type of machine learning that uses large neural networks with many layers to learn complex patterns from data.

It is particularly useful for things such as images, speech and language.

Example: A facial-recognition system learning patterns in people's faces.

4. Generative AI (GenAI)

Generative AI creates new content based on patterns it has learned from existing data.

It can generate text, images, music, video, code and other types of content.

Example: ChatGPT generating an explanation or an image-generation model creating a picture from a text description.

AI assistant's review of the diagram

I would ask an AI assistant:

"Explain my diagram showing the relationship between AI, ML, Deep Learning and Generative AI. Identify anything that could be misleading."

Review / correction

The main relationship is correct:

AI → includes ML → includes Deep Learning.

However, it would be misleading to say:

"Generative AI is always a subset of Deep Learning."

That's too absolute.

A better explanation is:

Generative AI is a category of AI systems whose purpose is to generate new content. Most modern Generative AI is powered by machine learning and deep learning, but GenAI describes the capability/function rather than simply being another level in the hierarchy.

Final submission version

AI is the broad field of making computers perform tasks that require intelligence.

ML is a method within AI where systems learn patterns from data.

Deep Learning is a type of ML that uses multi-layer neural networks to learn complex patterns.

Generative AI creates new content such as text, images, audio or code, and most modern GenAI is built using deep learning.

Mission 3 — Is It Really AI?

Classification table

System / Feature	Classification	Why?
Calculator	Rule-based / traditional software	It follows predefined mathematical rules. It does not learn patterns from data.
Temperature warning rule	Rule-based / traditional software	A fixed rule such as “if temperature > 40°C, show warning” produces the result.
Spam filter	ML-based AI	It can learn patterns from examples of spam and legitimate emails and use those patterns to classify new emails.
Document summariser	Generative AI	A modern AI summariser can understand the document and generate a new, shorter version in its own words.
Traffic ETA prediction	ML-based AI	It can learn from historical traffic, road conditions, time of day and other data to predict travel time.

1. An easy classification

Calculator — Rule-based software

This was easy to classify because a calculator follows **explicit mathematical rules**.

For example:

$$10 \times 5 \rightarrow 50$$

The calculator doesn't need to learn from previous calculations. It simply follows programmed instructions.

2. A difficult classification

Document summariser — Generative AI

This one can be difficult because not every document summariser necessarily uses generative AI.

A simple older summariser could select important sentences using predefined rules or statistical methods. A modern AI summariser, however, can understand the document and generate a new summary in different wording.

Therefore, for this mission, I classify a modern document summariser as **Generative AI**, while recognizing that the exact technology depends on the particular tool.

3. My own example

Netflix movie recommendation

Classification: ML-based AI

Netflix can use information such as what I watch, what I search for, what I rate, and what similar users watch to predict which movies or shows I might enjoy.

It is not simply following one fixed rule such as:

“If user watches action → recommend movie X.”

Instead, machine-learning models can identify patterns in large amounts of user and content data.

Short conclusion

Not everything that appears “smart” is AI. **A calculator and a fixed temperature warning can work entirely through explicit rules, while spam filtering and traffic prediction can learn patterns from data. Generative AI goes further by creating new content, such as a newly written document summary.** The key difference is whether the system is simply following programmed rules or using learned patterns/generative models.

Mission 4 — Make AI Explain Itself, Then Test It

1. How an LLM produces an answer to my prompt

When I send a prompt to an LLM, the process can be understood in these basic steps:

My prompt

↓

Text is divided into tokens

↓

The model processes the tokens and their relationships

↓

Transformer/self-attention uses the context to determine what is relevant

↓

The model predicts likely next token(s)

↓

New token is added to the response

↓

The process repeats

↓

Final answer

OpenAI explains that text is divided into tokens, which can be words, parts of words, characters, or punctuation. The model processes these tokens and generates output tokens.

2. Beginner explanation with a simple example

Imagine I ask:

“What is the capital of France?”

The LLM doesn't simply look up the answer from a human-like database in the way a search engine does.

Instead, it processes the prompt as tokens and uses patterns learned during training to predict an appropriate continuation.

Very simplified:

"What is the capital of France?"

↓

Understand context

↓

Predict likely next token

↓

"The"

↓

"capital"

↓

"of"

↓

"France"

↓

"is"

↓

"Paris"

The actual model is vastly more complicated than this example. It uses a Transformer architecture with attention mechanisms to process relationships between tokens and context.

3. My own explanation of how an LLM works

An LLM is a neural-network model trained on large amounts of data to learn patterns in language. When I give it a prompt, the text is converted into tokens. The model processes those tokens using its learned parameters and Transformer architecture, including attention mechanisms that help it consider relationships between parts of the context. It then predicts suitable output tokens and generates the response step by step.

It is important to understand that **“predicting the next token” does not mean the model is simply guessing randomly**. Its predictions are based on patterns encoded in a very large set of learned parameters.

4. Two important claims — checked with reliable sources

Claim 1: “LLMs work with tokens.”

Verified.

OpenAI's documentation states that tokens are the units its models use to process text. A token can represent a whole word, part of a word, a character, or punctuation. The model processes input tokens and generates output tokens.

Source: [OpenAI — Understanding and counting tokens](#)

Claim 2: “Transformers use attention to understand relationships between tokens.”

Verified.

Google's explanation of Transformers describes **self-attention** as a mechanism that determines how much different tokens affect the interpretation of one another. The original Transformer research introduced an architecture based on attention mechanisms.

Sources:

[Google for Developers — What's a large language model?](#)

[Google Research — Attention Is All You Need](#)

5. What the AI explained well

The AI explanation correctly helped me understand that:

An LLM does not produce an answer by simply following a fixed list of rules. It uses patterns learned during training and processes the context of my prompt before generating output.

The concept of **tokens + Transformer + attention + prediction** gives me a useful basic mental model without requiring me to understand all the mathematics behind neural networks.

6. What I had to clarify/correct

One thing that needs clarification is the statement:

“The LLM understands my question and then finds the answer.”

This can give a misleading picture.

A better explanation is:

The LLM processes the input and uses learned patterns and its model parameters to generate a likely sequence of tokens.

Also, saying:

“The LLM just predicts the next word.”

is too simplistic.

It predicts **tokens**, not necessarily complete words, and modern LLMs involve many layers of neural-network computation and attention mechanisms before each generated token is selected.

Final reflection

What I learned

I learned that an LLM does not work like a human who reads a question and consciously searches their memory for an answer. It converts text into tokens, processes their relationships through a neural network, and generates output token by token based on patterns learned during training. The Transformer and attention mechanism help the model use the context of the prompt.

My basic mental model

Prompt → Tokens → Transformer/Attention → Prediction → Output tokens → Answer

Mission 5 — Can AI Be Confidently Wrong?

1. Prompt I gave the AI

“In JavaScript, does the map() method modify the original array? Explain with a simple example.”

2. AI answer

The AI answered:

No. JavaScript's map() method does not modify the original array. It creates and returns a new array containing the results of applying the callback function to each element.

Example:

```
const numbers = [1, 2, 3];
const doubled = numbers.map(x => x * 2);
console.log(numbers); // [1, 2, 3]
console.log(doubled); // [2, 4, 6]
```

So the original numbers array remains unchanged.

3. Verification source

I checked the answer against **MDN (Mozilla Developer Network)**, a reliable technical reference for JavaScript.

MDN states that `map()` **creates a new array** containing the results of calling the provided function on every element. Its example also shows that the original array remains unchanged.

MDN's general Array documentation also lists `map()` among the methods that create new arrays rather than mutating the original array.

[MDN — Array.prototype.map\(\)](#)

4. Result

Correct — but with an important qualification.

The AI's main answer was **correct**.

`map()` itself does not modify the original array:

```
const numbers = [1, 2, 3];  
  
const result = numbers.map(x => x * 2);  
  
console.log(numbers);  
  
// [1, 2, 3]
```

However, there is an important detail: **the callback function can itself cause side effects or modify objects contained in the array**. MDN explicitly discusses this distinction.

For example:

```
const users = [{name: "John"}];  
  
const result = users.map(user => {  
  user.name = "Mike";  
  return user;  
});
```

```
console.log(users);  
  
// [{name: "Mike"}]
```

Here, `map()` didn't directly mutate the array structure, but the callback changed an object inside it.

So the more precise statement is:

`map()` creates a new array and does not itself mutate the original array, but the callback can still cause changes to objects or other external data.

5. What did my test prove?

My test **proved that the basic claim about `map()` is supported by the JavaScript documentation.**

However, it **did not prove that every possible use of `map()` leaves everything completely unchanged.**

That's because the callback can perform its own operations, including modifying objects or producing other side effects.

This distinction is important.

6. Lesson

The main lesson I learned is:

An AI answer can sound completely confident, but confidence is not evidence that the answer is correct.

Even when the AI gives a correct answer, I should still **verify important factual or technical claims using an appropriate reliable source.**

In this case, the AI was correct, but verification revealed a useful **qualification** that makes the explanation more precise.

Final takeaway

AI answer → Don't automatically trust it → Check a reliable source → Compare the evidence → Identify limitations → Then accept or correct the answer.

Mission 6 — Chatbot or Agent?

1. Key concepts

Concept	Simple explanation	Example
LLM (Large Language Model)	A model trained on large amounts of text that can understand and generate language. It can answer questions and produce text, but by itself it does not automatically take actions in external systems.	ChatGPT answering: "Explain machine learning."
AI Application	A complete software application that uses AI to perform a useful task. It may combine an LLM with a user interface, data, tools, databases, etc.	A customer-support chatbot that uses an LLM to answer customer questions.
RAG (Retrieval-Augmented Generation)	A system that first retrieves relevant information from a knowledge source and then gives that information to the model so it can generate a better answer.	Asking a company chatbot about its employee policy and having it retrieve the relevant policy document first.
Tool-using Assistant	An AI assistant that can use external tools or services to get information or perform tasks instead of only generating text.	An assistant that checks a calendar and tells you when you are free.

Concept	Simple explanation	Example
Agent	A system that can work toward a goal by deciding what steps to take, using tools, observing results, and continuing until the task is completed or it needs human input.	“Find a suitable flight, compare options, and prepare the best itinerary.”

2. Simple flow diagram



The important difference

A basic chatbot might do:

User → LLM → Answer

A tool-using assistant can do:

User → LLM → Tool/Retrieval → Result → LLM → Answer/Action

An **agent** can go further:

Goal → Decide step → Use tool → Observe result → Decide next step → Use another tool → Complete goal

3. Comparison

Feature	LLM	AI Application	RAG	Tool-using Assistant	Agent
Generates language	✓	Usually	✓	Usually	Usually
Retrieves external information	Not necessarily	Depends	✓	✓	Often
Uses external tools	✗ by itself	Depends	Not necessarily	✓	✓
Takes actions	✗ by itself	Depends	Usually no	✓	✓
Works toward a multi-step goal	✗ by itself	Depends	Usually no	Sometimes	Yes
Decides what steps/tools to use	✗	Depends	Limited	Some	Core capability

4. Everyday example of an agentic workflow

Example: Planning a trip

Suppose I tell an AI assistant:

“Plan my trip to Mumbai for next weekend and find a suitable hotel.”

A simple chatbot could respond with general travel suggestions.

But an **agentic system** could potentially:

User gives goal



AI understands requirements



Check travel options



Compare times/prices



Search hotel options



Compare hotels



Create an itinerary



Present the best options



Ask user for approval before booking

5. What changes from chatbot → agent?

Chatbot

“Ask me something and I'll give you an answer.”

Tool-using assistant

“Ask me something and I can use a tool to get information or perform an action.”

Agent

“Give me a goal and I can determine and execute a sequence of steps using available tools to work toward that goal.”

Mission 7 — What Actually Runs AI?

1. What is a CPU and what is it good at?

CPU (Central Processing Unit) is the general-purpose processor in a computer. It is designed to handle many different types of tasks and is good at **sequential operations, decision-making, operating-system tasks, and general-purpose computing**.

A CPU usually has a smaller number of powerful cores compared with a GPU.

Example: Running an operating system, opening applications, handling files, and executing normal program instructions.

2. What is a GPU and why is it useful for AI workloads?

GPU (Graphics Processing Unit) is a processor designed to perform many calculations in parallel.

AI models perform enormous numbers of mathematical operations, especially **matrix and vector calculations**. GPUs can perform many of these operations simultaneously, making them very useful for AI.

Example: During neural-network training, thousands or millions of numerical calculations can be performed in parallel on a GPU.

Simple idea:

CPU: fewer powerful workers → good for varied/general tasks

GPU: many parallel workers → good for large amounts of similar calculations

3. What is an NPU / AI accelerator?

NPU (Neural Processing Unit) or AI accelerator is specialised hardware designed specifically to speed up AI and machine-learning operations.

Modern devices may include dedicated AI hardware because running AI models can require substantial computation. An accelerator can perform certain AI operations more efficiently and sometimes with lower power consumption than using a general-purpose CPU alone.

Examples: Smartphone AI features such as speech recognition, image processing, camera effects, and on-device AI can use specialised AI hardware.

4. What does parallel computation mean?

Parallel computation means performing multiple calculations at the same time instead of doing everything one calculation after another.

For example, imagine multiplying 1,000 numbers.

Sequential:

Calculation 1



Calculation 2



Calculation 3



...

Calculation 1000

Parallel:

Calculation 1 —|

Calculation 2 —|

Calculation 3 —| —→ Results

Calculation 4 —|

Calculation 5 —|

...

This is one reason GPUs are particularly useful for AI workloads.

5. Why does AI depend so heavily on compute and memory?

AI models contain **large numbers of parameters** and perform huge numbers of mathematical operations.

Two resources are especially important:

Compute

Compute is needed to perform operations such as:

- matrix multiplication
- additions
- multiplications
- neural-network calculations

More available compute can allow these calculations to be performed faster.

Memory

Memory is needed to store things such as:

- model parameters
- input data
- intermediate calculations
- activations
- training information

Large AI models can require very large amounts of memory.

Simple analogy

Think of an AI system as a factory:

Compute = workers doing the calculations

Memory = workspace/storage holding the materials

You need both for the factory to operate effectively.

6. Training vs inference from a hardware/computation viewpoint

	Training	Inference
What happens?	Model learns its parameters from training data.	Trained model produces an output for new input.
Computation	Usually extremely large	Usually smaller per request

	Training	Inference
Hardware demand	Very high	Can range from small to very high
Memory	High, because training stores intermediate information and model data	Generally lower, although large models can still require substantial memory
Example	Training a new language model	Asking ChatGPT a question

Simple explanation

Training = teaching the model.

The system repeatedly processes data and adjusts the model's parameters.

Inference = using what the model has learned.

The parameters are used to process new input and produce an output.

7. Simple picture: what actually runs AI?



What this means

When I use an AI application, the application communicates with an AI model. Software frameworks help execute the model, and the actual calculations are performed by hardware such as CPUs, GPUs, or specialised accelerators, using memory to store the model and working data.

8. Real AI workload

Example: Training a Large Language Model

A good example of a demanding AI workload is **training an LLM**.

What happens?

The model processes enormous amounts of training data and performs huge numbers of mathematical operations. Its parameters are repeatedly adjusted during training.

Useful hardware

GPU / AI accelerator → highly useful

Why?

Because neural-network training involves enormous amounts of parallel mathematical computation, particularly matrix operations.

A training system might therefore use:

Large training dataset



GPU cluster



Parallel calculations



Update model parameters



Repeat many times



Trained model

CPUs are still useful for tasks such as **data preparation, coordination, operating-system functions and other general-purpose work**, but GPUs or specialised accelerators are particularly important for the heavy neural-network computation.

Mission 8 — Where Could AI Help Your VLSI Track?

For this mission, I'll choose **Physical Design (PD)** because it is a major VLSI area where AI can help with repetitive analysis, optimization, prediction, and debugging.

Small table

VLSI area	One task	How AI might help	Why human/domain knowledge still matters
Physical Design (PD)	Timing optimization / timing-closure analysis	AI can analyze large amounts of timing data, identify patterns causing timing violations, and suggest or prioritize optimization strategies.	A VLSI engineer must understand constraints, timing paths, physical effects, PVT corners, congestion, power, and design requirements to judge whether an AI suggestion is actually appropriate.

Short explanation

AI can help Physical Design engineers by quickly analyzing large numbers of timing paths and identifying patterns that may contribute to setup/hold violations, helping engineers prioritize optimization and reduce repetitive analysis.

However, human VLSI knowledge still matters because timing closure involves trade-offs between timing, area, power, congestion, routing, and design constraints, and an AI system may not understand the full design intent or practical consequences of a change.

Why this is a good example

The important point from Mission 8 is:

AI is not the VLSI domain. AI is a capability that can be applied to VLSI.

For example:

AI capability



Pattern recognition / prediction / optimization



Physical Design



Timing closure



Better engineering workflow

So I am **not saying** “AI replaces a Physical Design engineer.”

I'm saying:

AI can assist a Physical Design engineer with tasks that involve large amounts of data, repetition, prediction, and optimization, while the engineer provides the domain knowledge and final judgment.

Final submission paragraph

I chose Physical Design (PD), specifically timing-closure analysis. AI could help analyze large numbers of timing paths and identify patterns behind timing violations, allowing engineers to prioritize optimization work and reduce repetitive analysis. However, human VLSI knowledge remains essential because timing closure involves complex trade-offs among timing, power, area, congestion, routing, and design constraints, so an engineer must evaluate whether an AI-generated suggestion is technically correct and suitable for the design.

Mission 9 — Build Your Own AI-Use Rule

My 5-Rule AI Working Agreement

As a VLSI engineering student, I will use AI as a **learning and productivity tool**, but I will remain responsible for understanding and verifying my work.

Rule 1 — Always verify important technical information

I will **not automatically trust an AI answer**, especially for technical concepts, calculations, code, VLSI design information, or engineering decisions. I will verify important claims using reliable sources such as official documentation, textbooks, research papers, or trusted technical references.

Why: In Mission 5, I learned that an AI can give a confident answer that may be incomplete or require qualification. Therefore, **confidence is not evidence of correctness**.

Rule 2 — Never share confidential or proprietary information

I will not put **confidential company information, proprietary designs, unreleased project information, passwords, private data, source code, or sensitive documents** into an AI tool unless I am explicitly authorized to do so.

I will use anonymized or fictional examples when possible.

Rule 3 — I remain responsible for my final work

I will treat AI as an **assistant, not the owner of my work**.

If AI helps me write code, solve a problem, explain a concept, or prepare an answer, I will understand and review the result before submitting or using it.

If I cannot explain or defend the result myself, I should not blindly submit it.

Rule 4 — Use AI to learn, not just to copy

When learning engineering concepts, I will first try to understand the problem and then use AI to:

- explain difficult concepts,
- provide examples,
- identify mistakes,

- compare approaches,
- help debug,
- or suggest ways to investigate further.

I will avoid using AI simply to generate an answer that I do not understand.

Rule 5 — Keep track of important AI-assisted work

When AI contributes significantly to an engineering task, I will keep a record of the **prompt, important AI suggestions, sources used for verification, and changes I made.**

This helps me distinguish between:

What AI suggested vs. what I actually verified and decided.

Justification using Missions 4 and 5

From Mission 4

I learned that an LLM generates responses by processing tokens and using patterns learned during training. This means an AI response is **generated by a model**, not automatically guaranteed to be factually correct.

From Mission 5

I tested an AI-generated technical answer about JavaScript's `map()` function. The main answer was correct, but verification revealed an important qualification about how the callback can modify objects.

This demonstrated that:

Even a correct-sounding AI answer should be checked when accuracy matters.

Therefore, my most important rule is:

“Use AI to accelerate my work, but verify important information before I trust or submit it.”

My 5-Rule AI Working Agreement — Final Version

1. Verify important technical claims before trusting them.
2. Never share confidential or proprietary information without authorization.
3. Take responsibility for my final engineering work.
4. Use AI to improve my understanding, not simply to copy answers.
5. Keep track of important AI assistance and my own verification.

Activity B – Python program

Activity B - Python program

```
def add_numbers(a, b):  
    result = a + b  
    return result  
  
num1 = int(input("Enter the first number: "))  
num2 = int(input("Enter the second number: "))  
answer = add_numbers(num1, num2)  
print("The sum is:", answer)
```

Final Reflection

1. What is the most useful thing you learned this week?

The most useful thing I learned is that **AI should be treated as a tool that helps me think and work, not as an authority that is always correct**. I learned how AI, ML, deep learning, Generative AI, LLMs, agents, and AI hardware are connected.

2. What did you initially misunderstand?

Initially, I thought that **anything that behaves intelligently must be AI**. I also had a simplified understanding that AI simply “knows” answers. I now understand that some systems are just rule-based, while AI systems can learn patterns from data, and LLMs generate responses based on learned patterns.

3. What did AI help you do?

AI helped me **understand difficult technical concepts in simple language**, organize my answers, create diagrams and examples, and connect AI concepts with my VLSI studies. It also helped me identify areas where I needed to investigate further.

4. Where did you need to verify or correct AI?

I needed to verify AI when dealing with **technical and factual claims**. For example, when investigating how an LLM produces an answer, I checked claims about tokens and Transformer/attention mechanisms using reliable sources. I also learned that an AI answer can be broadly correct but still need an important qualification.

5. What is one question you want answered on Saturday?

How can AI be practically used in VLSI design and verification without replacing the engineer's technical judgment?